Homework 6 - Written Responses and Plots

Jordan Sheppard

September 25, 2024

[1]:
```
import numpy as np
from matplotlib import pyplot as plt
from MultiDimensionalBasisFunctions import PlotTwoDimensionalParentBasisFunction
from UniDimensionalXMap import PlotXMap

plt.rcParams['figure.figsize'] = (5.5, 5.5)
plt.rcParams['figure.dpi'] = 80
```

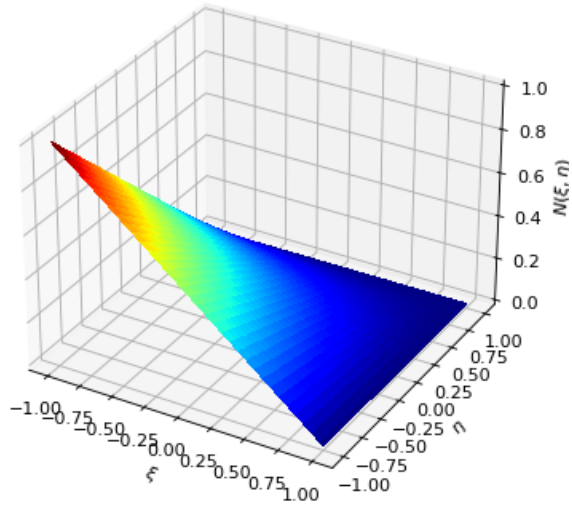
Problem 1

Plot two-dimensional basis function of the following polynomial degrees over the interval $[-1, 1]$ using the `PlotTwoDimensionalParentBasisFunction` code given to you.

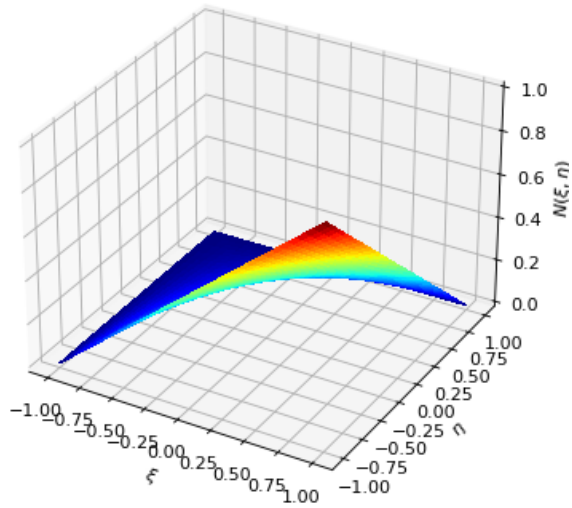
- Degree $[1, 1]$ polynomials

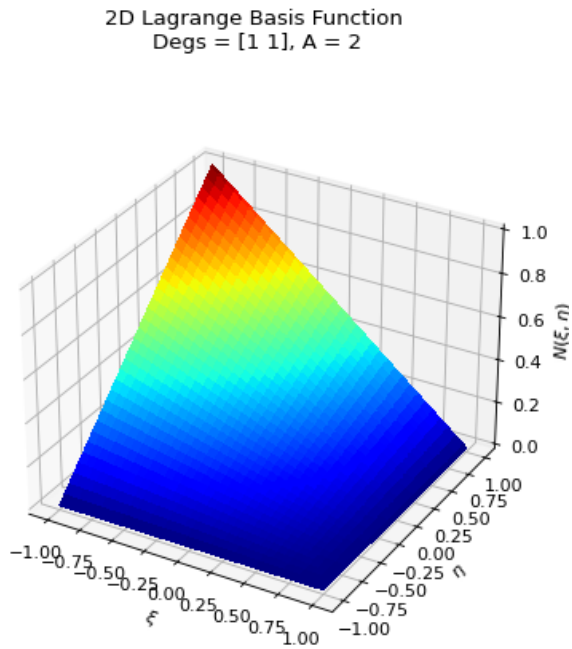
[2]:
```
degs = np.array([1, 1])
num_funcs = np.prod(degs + np.ones_like(degs))
for A in range(num_funcs):
    PlotTwoDimensionalParentBasisFunction(A, degs, contours=False)
```

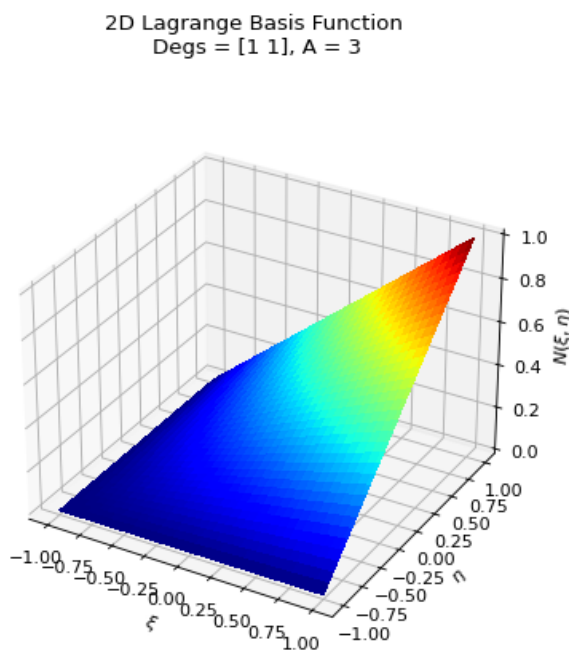
2D Lagrange Basis Function
Degs = [1 1], A = 0



2D Lagrange Basis Function
Degs = [1 1], A = 1

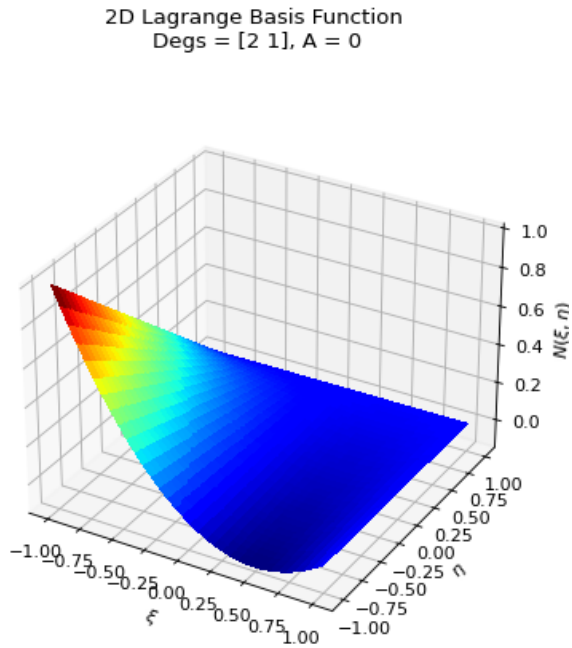


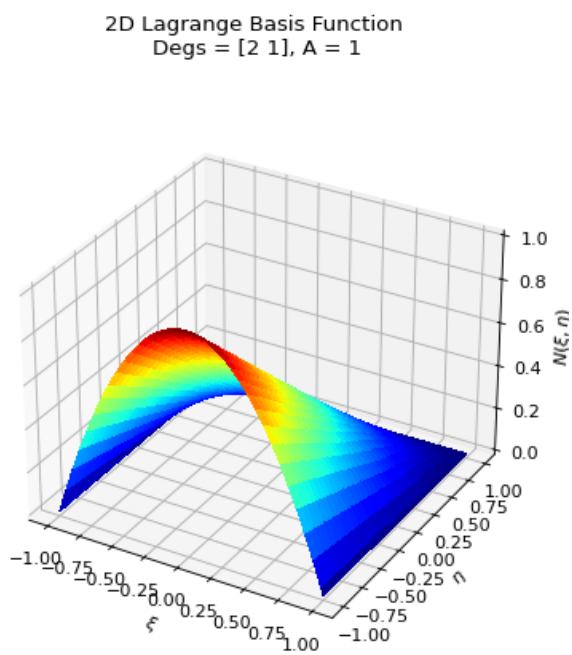


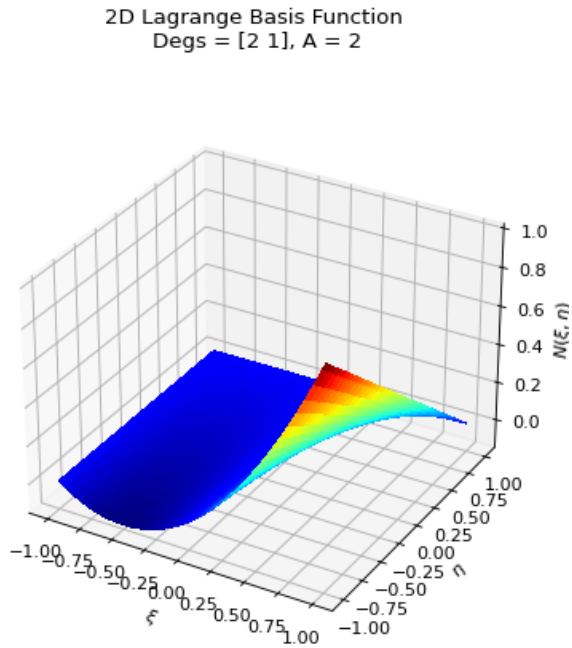


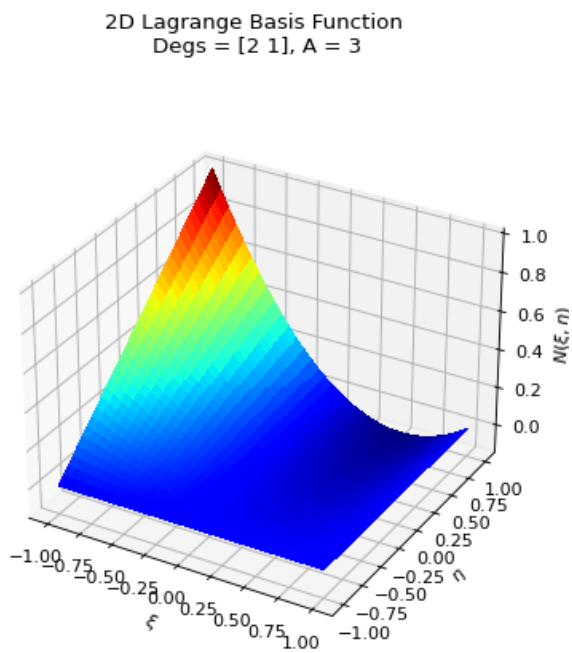
- Degree $[2, 1]$ polynomials

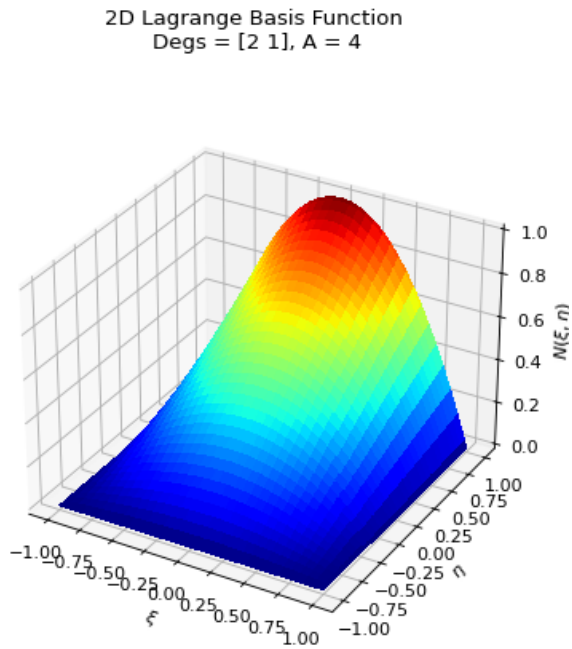
[3]:
```
degs = np.array([2, 1])
num_funcs = np.prod(degs + np.ones_like(degs))
for A in range(num_funcs):
    PlotTwoDimensionalParentBasisFunction(A, degs, contours=False)
```

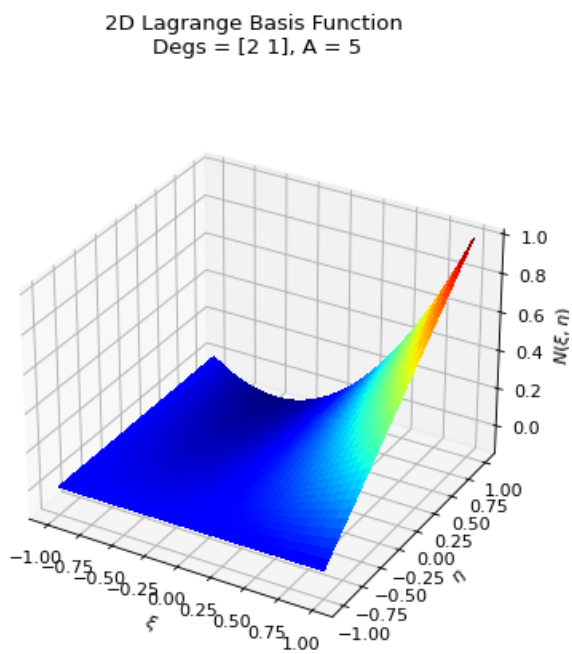





2D Lagrange Basis Function
Degs = [2 1], A = 2
1.0
0.8
0.6
0.4
0.2
0.0
N(ξ, η)
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
ξ
η

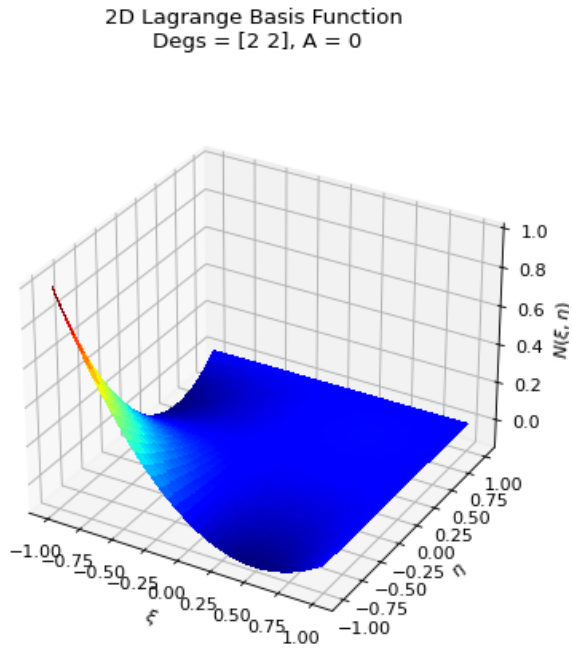

2D Lagrange Basis Function
Degs = [2 1], A = 3
1.0
0.8
0.6
0.4
0.2
0.0
N(ξ, η)
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
ξ
η

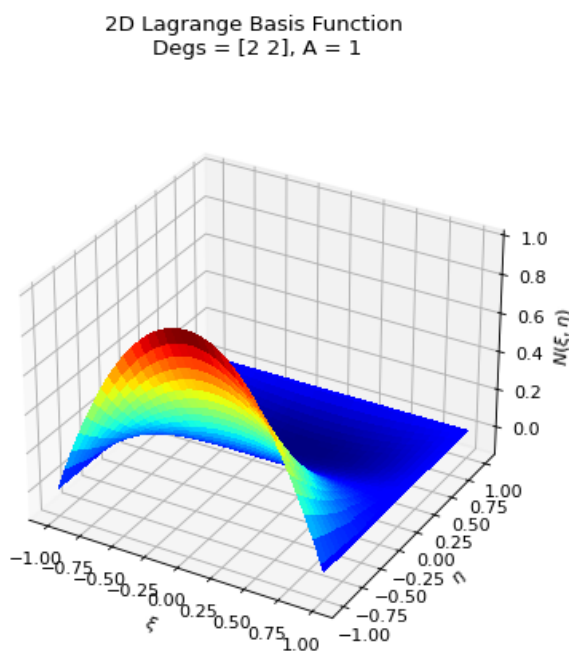




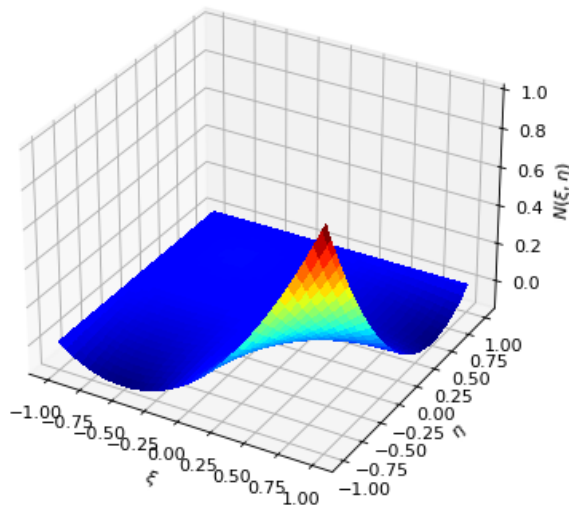
- Degree $[2, 2]$ polynomials

[4]:
```
degs = np.array([2, 2])
num_funcs = np.prod(degs + np.ones_like(degs))
for A in range(num_funcs):
    PlotTwoDimensionalParentBasisFunction(A, degs, contours=False)
```

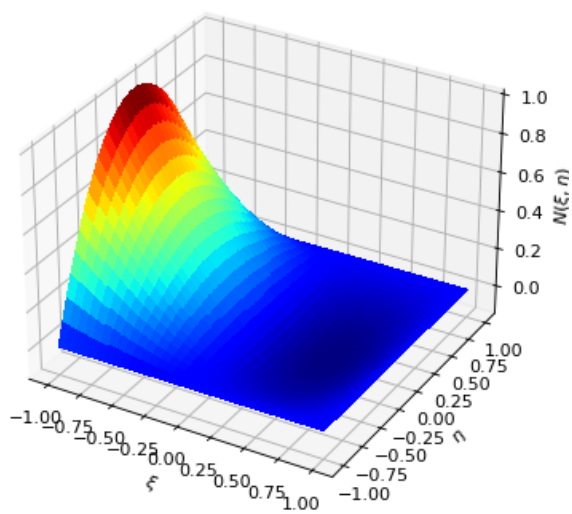




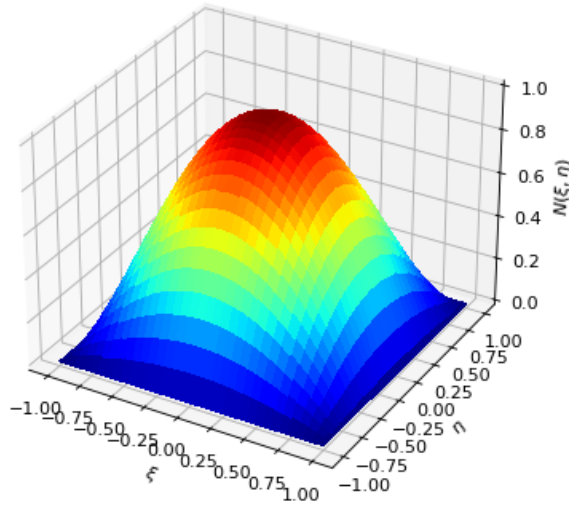
2D Lagrange Basis Function
Degs = [2 2], A = 2



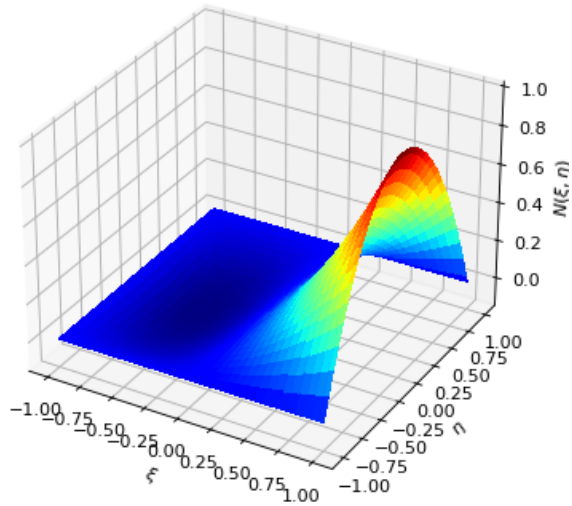
2D Lagrange Basis Function
Degs = [2 2], A = 3



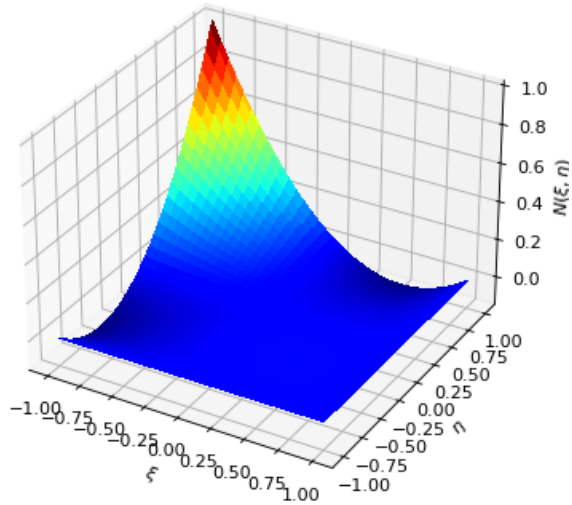
2D Lagrange Basis Function
Degs = [2 2], A = 4



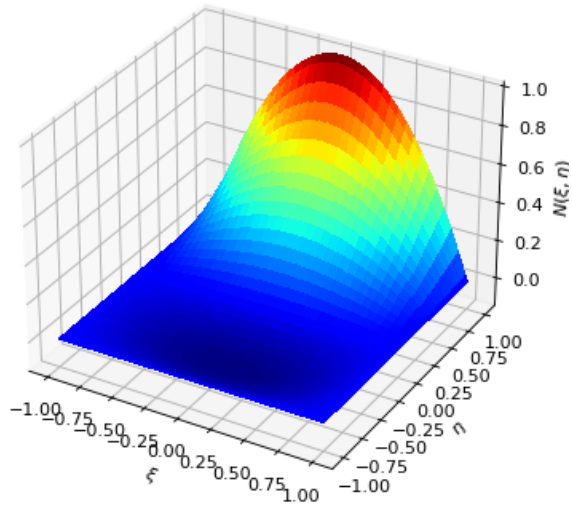
2D Lagrange Basis Function
Degs = [2 2], A = 5

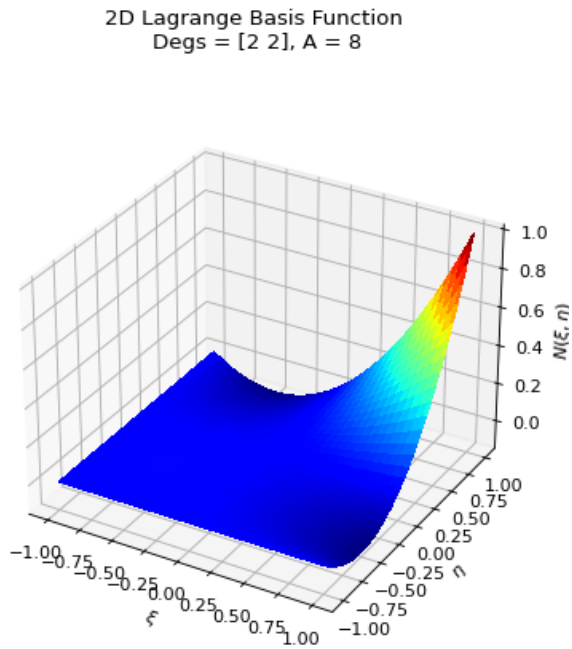


2D Lagrange Basis Function
Degs = [2 2], A = 6



2D Lagrange Basis Function
Degs = [2 2], A = 7

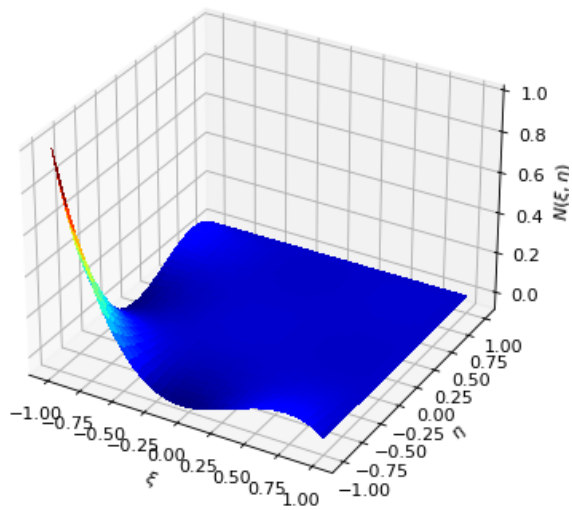




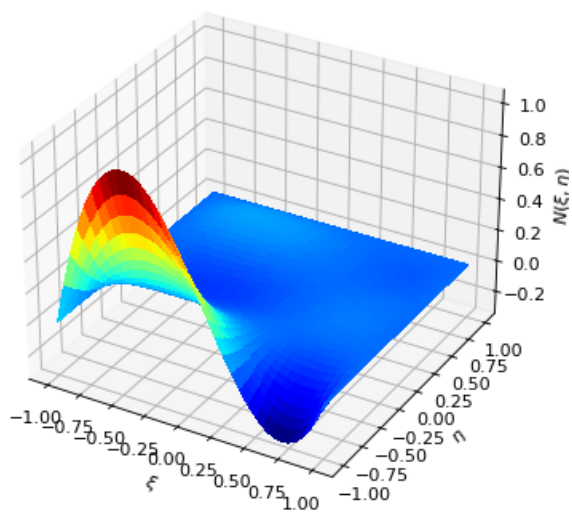
- Degree $[3, 3]$ polynomials

[5]:
```
degs = np.array([3, 3])
num_funcs = np.prod(degs + np.ones_like(degs))
for A in range(num_funcs):
    PlotTwoDimensionalParentBasisFunction(A, degs, contours=False)
```

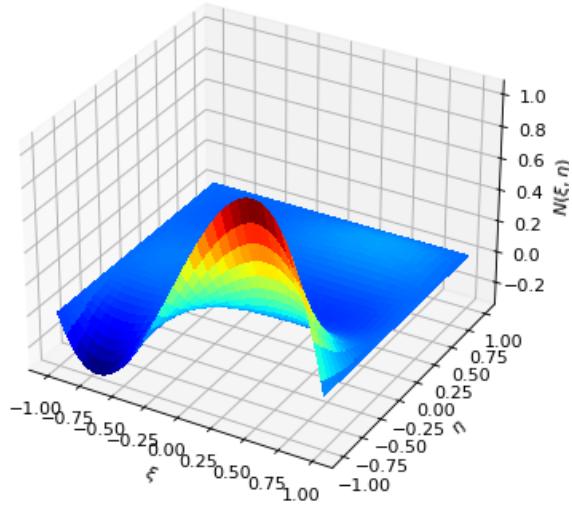
2D Lagrange Basis Function
Degs = [3 3], A = 0



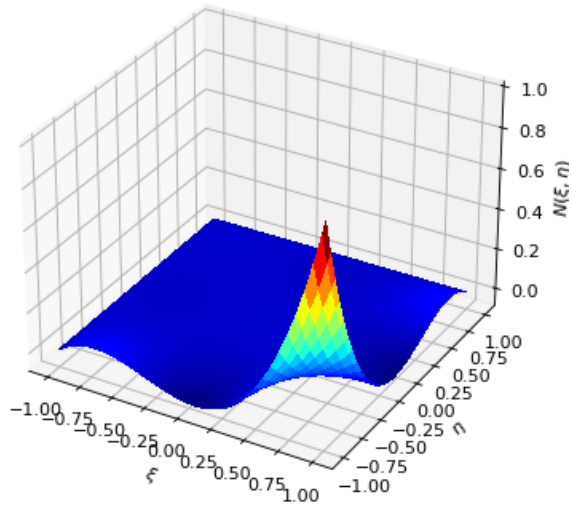
2D Lagrange Basis Function
Degs = [3 3], A = 1



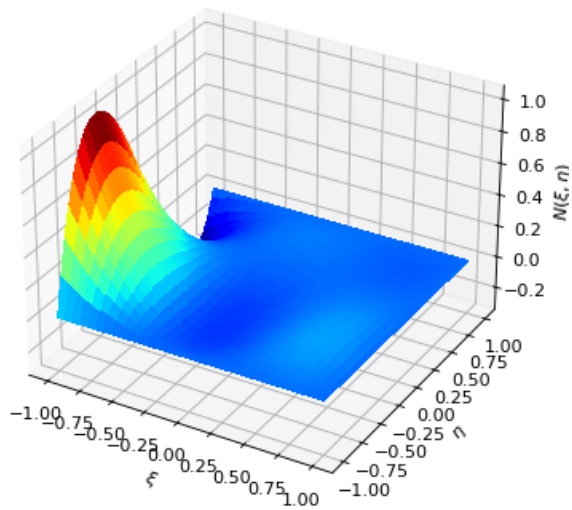
2D Lagrange Basis Function
Degs = [3 3], A = 2



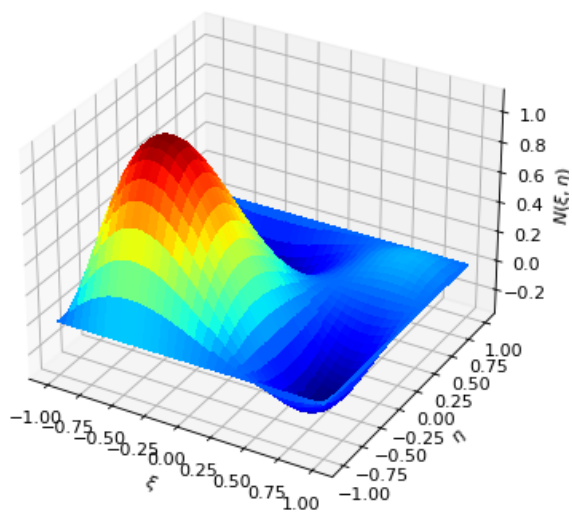
2D Lagrange Basis Function
Degs = [3 3], A = 3



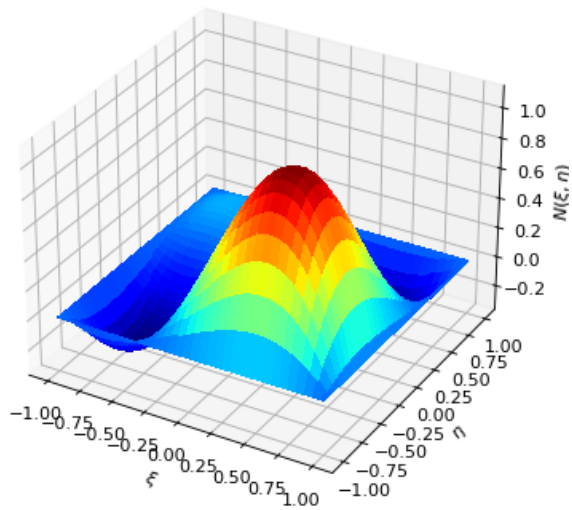
2D Lagrange Basis Function
Degs = [3 3], A = 4



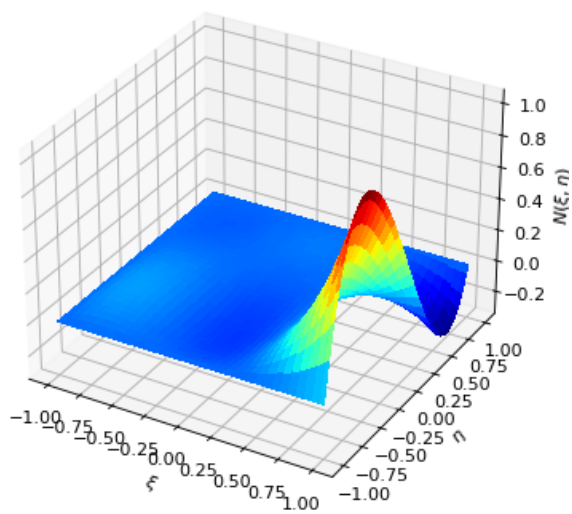
2D Lagrange Basis Function
Degs = [3 3], A = 5



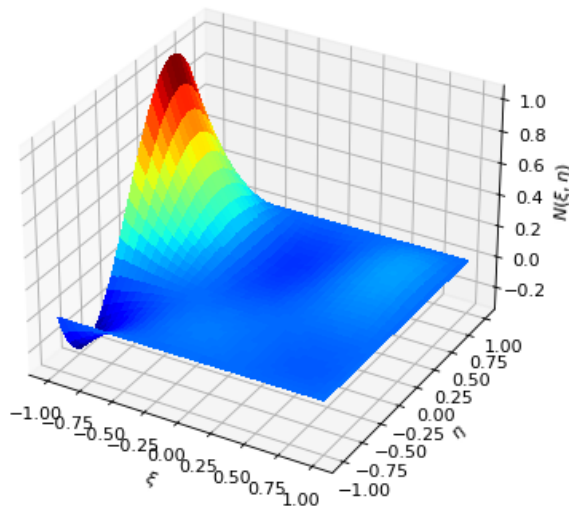
2D Lagrange Basis Function
Degs = [3 3], A = 6



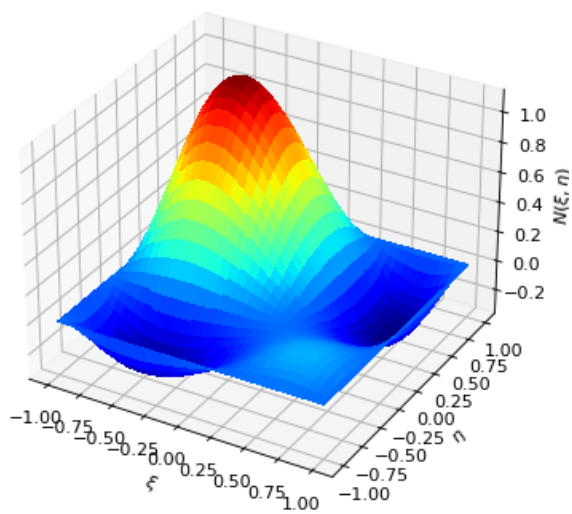
2D Lagrange Basis Function
Degs = [3 3], A = 7



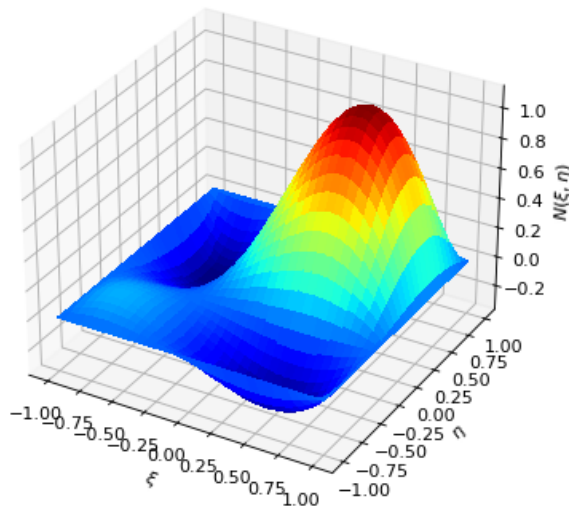
2D Lagrange Basis Function
Degs = [3 3], A = 8



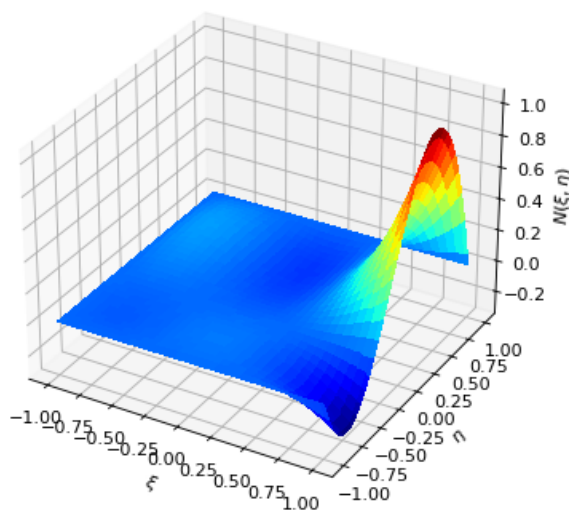
2D Lagrange Basis Function
Degs = [3 3], A = 9

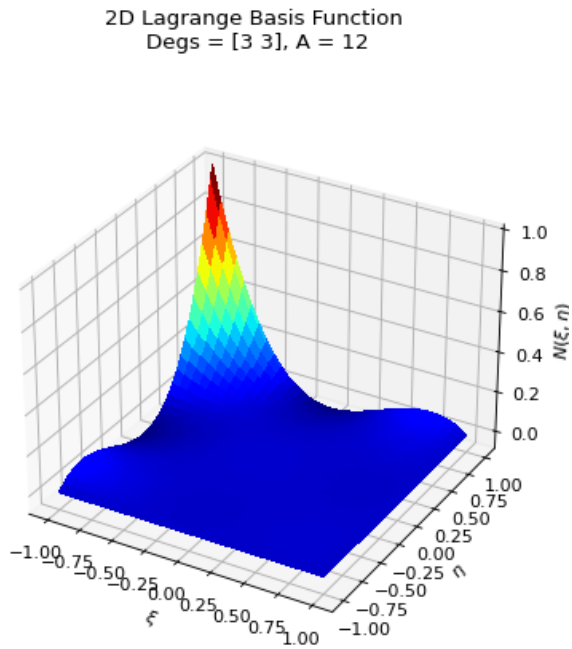


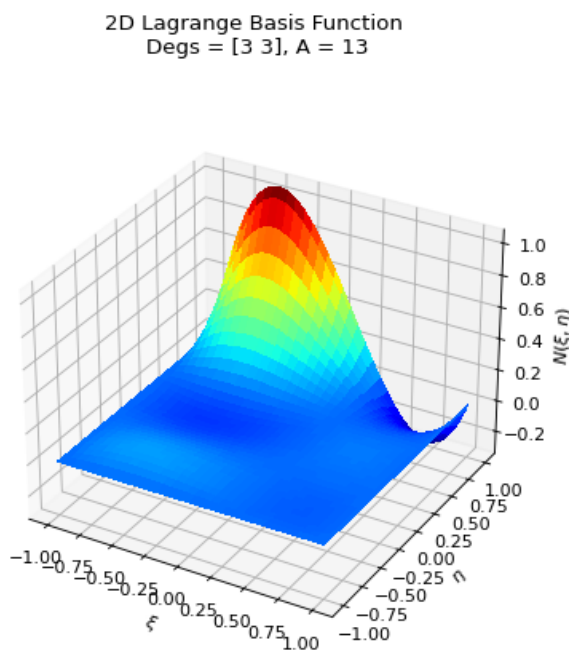
2D Lagrange Basis Function
Degs = [3 3], A = 10



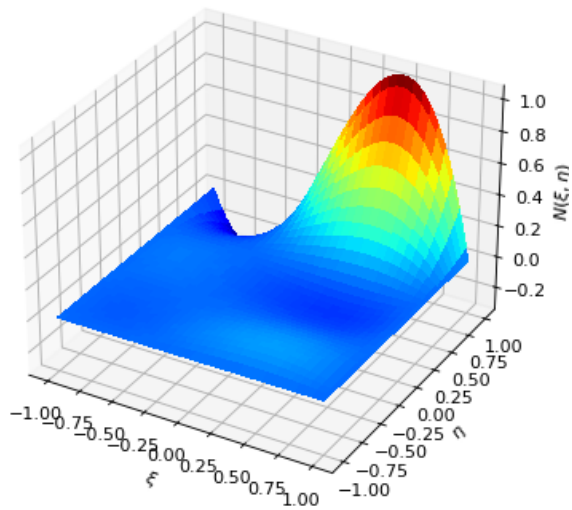
2D Lagrange Basis Function
Degs = [3 3], A = 11



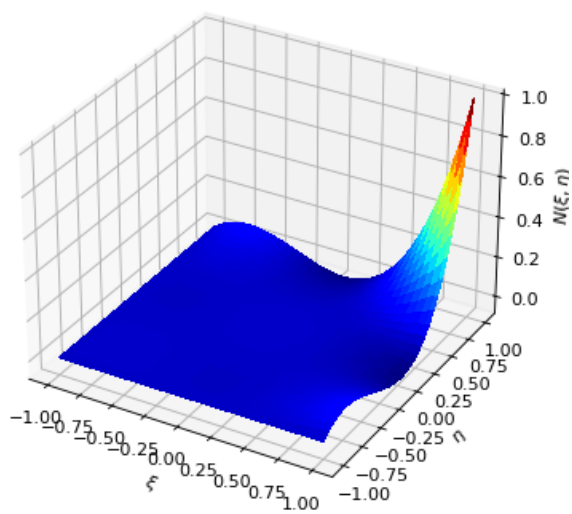

2D Lagrange Basis Function
Degs = [3 3], A = 12
1.0
0.8
0.6
0.4
0.2
0.0
N(ξ, η)
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
ξ
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
η


2D Lagrange Basis Function
Degs = [3 3], A = 13
1.0
0.8
0.6
0.4
0.2
0.0
−0.2
N(ξ, η)
−1.00
−0.75
−0.50
−0.25
0.00
0.25
0.50
0.75
1.00
ξ
1.00
0.75
0.50
0.25
0.00
−0.25
−0.50
−0.75
−1.00
η

2D Lagrange Basis Function
Degs = [3 3], A = 14



2D Lagrange Basis Function
Degs = [3 3], A = 15

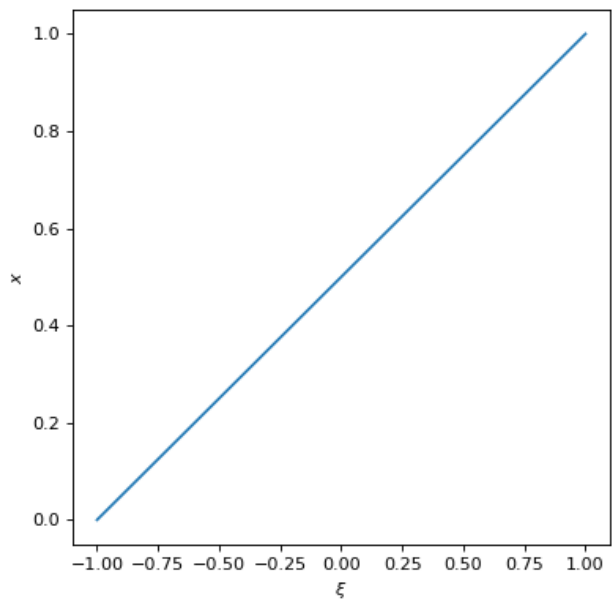


Problem 2

Map from the parent ξ-domain $[-1, 1]$ to a given spatial domain, given points in the ξ-domain and corresponding points in the spatial domain. Use the `PlotXMap` function from your code.

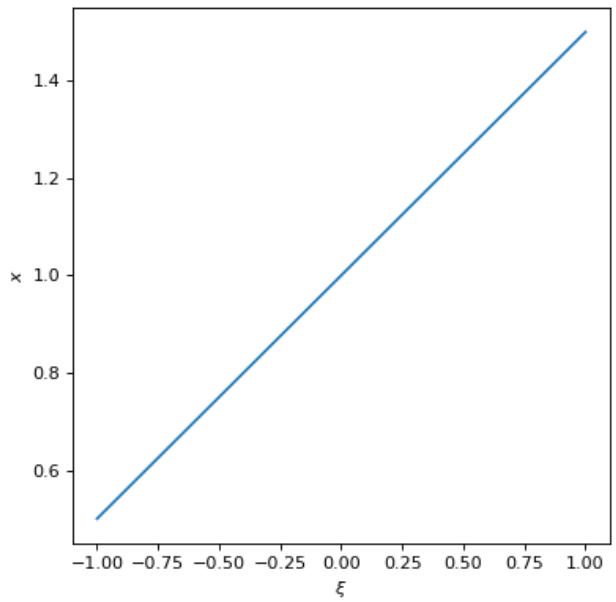
- $p = 1$, $\{x_a\} = \{0, 1\}, \{\xi_a\} = \{-1, 1\}$.

[6]:
```
deg = 1
interp_pts = np.array([-1, 1])      # xi values in parent/parametric domain
spatial_pts = np.array([0, 1])      # corresponding x values in parent domain
PlotXMap(deg, spatial_pts, interp_pts, contours=False)
```



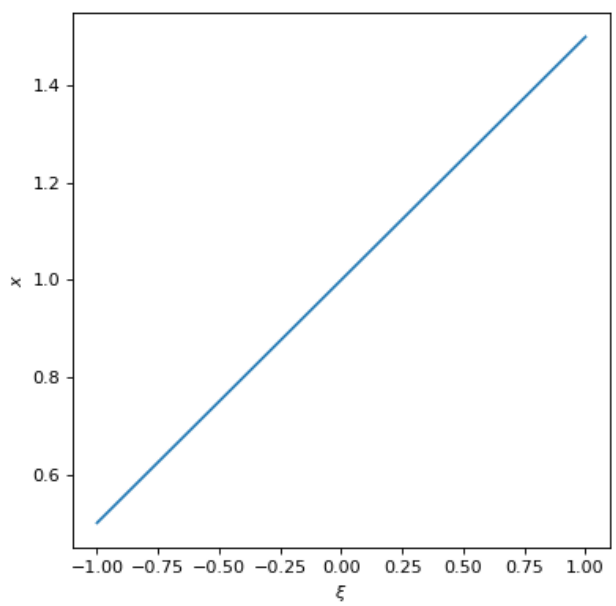
- $p = 2$, $\{x_a\} = \{0.5, 1, 1.5\}, \{\xi_a\} = \{-1, 0, 1\}$.

[7]:
```
deg = 2
interp_pts = np.array([-1, 0, 1])      # xi values in parent/parametric domain
spatial_pts = np.array([0.5, 1, 1.5])  # corresponding x values in parent domain
PlotXMap(deg, spatial_pts, interp_pts, contours=False)
```



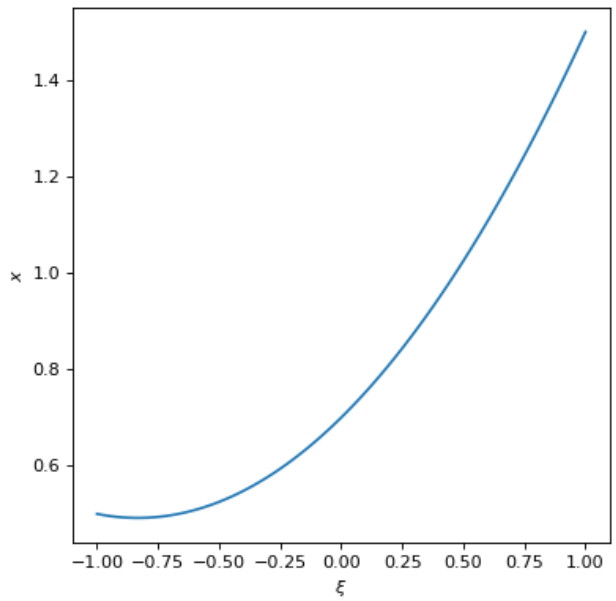
- $p = 2$, $\{x_a\} = \{0.5, 0.7, 1.5\}, \{\xi_a\} = \{-1, -0.6, 1\}$.

[8]:
```
deg = 2
interp_pts = np.array([-1, -0.6, 1])      # xi values in parent/parametric domain
spatial_pts = np.array([0.5, 0.7, 1.5])  # corresponding x values in parent␣
↪domain
PlotXMap(deg, spatial_pts, interp_pts, contours=False)
```



- $p = 2$, $\{x_a\} = \{0.5, 0.7, 1.5\}, \{\xi_a\} = \{-1, 0, 1\}$.

[9]:
```
deg = 2
interp_pts = np.array([-1, 0, 1])      # xi values in parent/parametric domain
spatial_pts = np.array([0.5, 0.7, 1.5])  # corresponding x values in parent␣
↪domain
PlotXMap(deg, spatial_pts, interp_pts, contours=False)
```



When is the mapping $x^e(\xi)$ linear?

We can see that the mapping is linear when the spacing in between the points in the parent ξ-domain corresponds to a linear stretch/squish of the spacing between the points in the spatial x-domain.

When is the map transitioning from the interval $[-1, 1]$ to the input spatial element in a manner that correctly transforms points for Gaussian quadrature?

We want the mapping $x^e(\xi)$ to be a linear map. So all but the last one of the above examples are satisfactory, while the last example is not.